

Cypher en profondeur

du cahier des charges à la requête — NOVANEST (Scénario 2)

Joseph Azar

Maître de conférences, UMLP · chercheur, FEMTO-ST

MG74 — Théorie des graphes & Neo4j · CFAI Exincourt, 8 juin 2026

Au programme (Scénario 2)

1. Le client **NOVANEST** & le cahier des charges
2. Rappel express + le **modèle e-commerce**
3. **Construire & modifier** le graphe : MERGE, propriétés, **CRUD**
4. **Interroger** : agrégations & tri (sum, avg, ORDER BY)
5. Aller plus loin : DISTINCT, OPTIONAL MATCH, hiérarchie, **recommandation**
6. Climax : l'**itinéraire de livraison** (shortestPath + reduce)

100 % graphe — une nouvelle entreprise, un nouveau besoin.

Rappel pratique

- **Suivez la démo** : je projette Neo4j Desktop (Browser localhost:7475).
- **Installation** de Neo4j Desktop : à la pause / chez vous / en autonomie (Windows · Linux · macOS) — pas pendant la séance.
- **Tous les supports** (slides PDF + Markdown + scripts Cypher) : **dépôt GitHub** fourni à la fin — vous refaites tout tranquillement.

Partie 1

Le client NOVANEST & le cahier des charges

Le client : NOVANEST SAS

Place de marché en ligne (high-tech & maison).
Elle a déjà le catalogue, les clients et l'historique d'achats — elle veut un **modèle 100 % graphe** (Neo4j).

- **Vendre plus** : recommander les bons produits.
- **Piloter** : best-sellers, notes, paniers (agrégats).
- **Livrer mieux** : l'itinéraire le moins cher / le plus rapide.

Trois familles de questions

Recommander (qui ressemble à qui ?)

Naviguer la hiérarchie de catégories

Router la livraison (chemin pondéré)

Le cahier des charges (extraits)

FR1 produit (nom, prix, marque, catégorie).

FR2 hiérarchie de catégories.

FR3 achat = relation **avec données** (qty, date, prix unitaire).

FR4 avis (étoiles, date).

FR5 best-sellers / note moyenne / dépense par client.

FR6 catégories achetées par un client (sans doublon).

FR7 produits d'une catégorie **et de ses sous-catégories**.

FR8 recommandation (exclure le déjà-acheté).

FR9 itinéraire de livraison le moins cher / le plus rapide.

△ FR7, FR8, FR9 : profondeur variable, recommandation, chemin pondéré. (on ouvre le PDF du CDC)

Partie 2

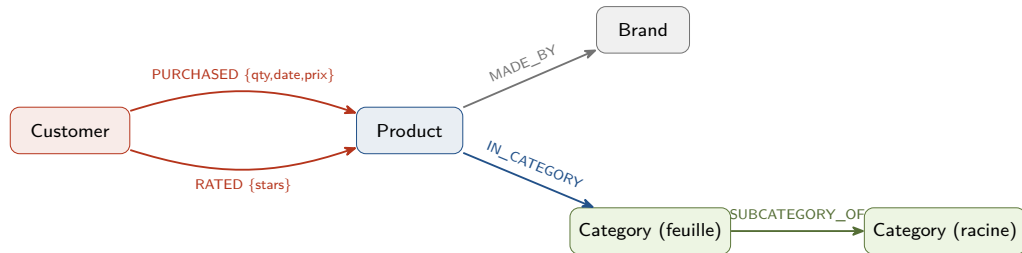
Rappel express & le modèle e-commerce

Rappel du Scénario 1 (30 secondes)

- Un **noeud** (), un **label** (:Product), une **propriété** {price:699}.
- Une **relation** orientée et typée -[:PURCHASED]->; elle peut **porter des propriétés**.
- Un **motif** se lit comme un dessin ; * = profondeur variable.
- Aujourd'hui on ajoute : MERGE, les **agrégations**, ORDER BY, DISTINCT, OPTIONAL MATCH, WITH, reduce.

Nouveau réflexe : CREATE crée toujours ; MERGE = « trouve ou crée » (idempotent).

Le modèle e-commerce (volet commercial)



+ volet logistique : (Product)-[:STOCKED_AT]->(Warehouse), (Customer)-[:LIVES_IN]->(City),
(Warehouse|Hub)-[:SHIPS_TO {hours,cost}]->(Hub|City) — détaillé en Partie 6.

Le jeu de données

Entité	Nb	Entité	Nb
Clients	11	Marques	3
Produits	13	Entrepôts	2
Catégories (2 niveaux)	6	Hubs	3
		Villes	3

41 nœuds. Assez petit pour tout visualiser, assez riche pour des agrégats et une recommandation non triviaux. (Un client, *Kenza*, n'a aucun achat — exprès.)

Partie 3

Construire & modifier le graphe (CRUD)

CREATE vs MERGE : l'idempotence

```
1 // CREATE cree TOUJOURS (relancer = doublon)
2 // MERGE = "trouve ou cree" -> relancable sans doubler
3 MERGE (b:Brand {name:'Nordia'}); // 1re fois : cree. 2e fois : ne fait rien.
```

Convention : MERGE sur la **clé naturelle**, puis SET le reste. On construit le graphe en **3 étapes** : noeuds
→ relations → relations à propriétés.

▷ [démonstration Neo4j](#)

Étape A — les noeuds (MERGE + SET)

```
1 MERGE (p:Product {id:1}) SET p.name='Smartphone Aura X', p.price=699
2 MERGE (c:Category {name:'Smartphones'}) SET c.level='feuille'
3 MERGE (u:Customer {id:1}) SET u.name='Alice';
```

On crée d'abord toutes les « choses » : marques, catégories, produits, clients + la couche logistique (entrepôts, hubs, villes). (*Extrait — le fichier `build.cypher` contient tout.*)

▷ [démonstration live Neo4j](#)

Étape B — les relations structurelles

```
1 MERGE (cSmart:Category {name:'Smartphones'})-[:SUBCATEGORY_OF]->
2       (cElec:Category {name:'Electronique'})
3 MERGE (p:Product {id:1})-[:IN_CATEGORY]->(cSmart)
4 MERGE (p)-[:MADE_BY]->(:Brand {name:'Nordia'});
```

Les liens *sans* données : la catégorie d'un produit, sa marque, la hiérarchie, le client→ville, le produit→entrepôt.

▷ [démonstration live Neo4j](#)

Étape C — les relations *avec propriétés*

```
1 MATCH (u:Customer {id:1}), (p:Product {id:1})
2 MERGE (u)-[:PURCHASED {date:'2026-06-01', qty:1, unitPrice:699}]->(p);
3 // idem RATED {stars,date} et la logistique SHIPS_TO {hours,cost}
```

L'achat est un lien qui porte ses données (qty, date, prix) — impossible proprement sur une clé étrangère SQL. C'est ça, « la relation est un objet à part entière ».

▷ **démo live Neo4j**

On charge bloc par bloc, puis on vérifie

```
1 // coller les blocs de build.cypher dans l'ordre, puis :  
2 MATCH (n) RETURN count(n);    // -> 41  
3 MATCH (n) RETURN n;           // le graphe se dessine
```

MERGE étant idempotent, relancer un bloc laisse **41** nœuds (aucun doublon). Astuce Browser : légende
→ Caption = name.

▷ [démonstration live Neo4j](#)

CRUD : les 4 opérations sur le graphe

	Opération	Clause Cypher
Create	créer	CREATE / MERGE
Read	lire / interroger	MATCH ...RETURN
Update	mettre à jour	SET (et REMOVE)
Delete	supprimer	DELETE / DETACH DELETE

On a déjà fait **C** (le build). Voyons **U** et **D** sur un produit « jetable » (id 14) qu'on supprimera ensuite — le graphe revient à 41 nœuds.

Ajouter (C) & mettre à jour (U)

```
1 // AJOUTER un produit jetable + le relier (MERGE = pas de doublon si on relance)
2 MERGE (p:Product {id:14}) SET p.name='Tablette NoteTab', p.price=329
3 MERGE (p)-[:IN_CATEGORY]->(:Category {name:'Ordinateurs'})
4 MERGE (p)-[:MADE_BY]->(:Brand {name:'Nordia'});
5
6 // METTRE A JOUR : modifier une propriete / en AJOUTER une nouvelle
7 MATCH (p:Product {id:14}) SET p.price = 299, p.enPromo = true RETURN p;
```

SET modifie une propriété existante *ou* en crée une (enPromo). REMOVE p.enPromo l'enlèverait.

▷ démo live Neo4j

Supprimer (D) : DELETE vs DETACH DELETE

```
1 // Supprimer une RELATION (ex. retirer le lien marque du produit 14)
2 MATCH (:Product {id:14})-[r:MADE_BY]->(:Brand) DELETE r;
3
4 // Supprimer un NOEUD encore relie :
5 MATCH (p:Product {id:14}) DELETE p;           // ERREUR : il a encore des relations
6 MATCH (p:Product {id:14}) DETACH DELETE p;     // OK : enleve le noeud ET ses relations
```

DELETE seul **refuse** de supprimer un nœud encore relié (sécurité). DETACH DELETE enlève le nœud **et** ses relations. Ensuite MATCH (n) RETURN count(n); → **41**.

▷ **démo live Neo4j**

Partie 4

Interroger : agrégations & tri

Q1 — Lister les smartphones

```
1 MATCH (p:Product)-[:IN_CATEGORY]->(:Category {name:'Smartphones'})
2 RETURN p.name, p.price ORDER BY p.price DESC;
```

Motif simple + filtre sur la catégorie reliée + tri. Attendu : Aura X 699, Pixel-N 599, Aura Mini 499.

▷ **démo live Neo4j**

Q2 — Produits d'une marque

```
1 MATCH (p:Product)-[:MADE_BY]->(:Brand {name:'Lumio'})
2 RETURN p.name, p.price ORDER BY p.name;
```

Suivre la flèche = la « jointure » disparaît. Attendu (5) : BassPro, Clavier MecaType, Ecouteurs AirLite, Grille-pain ToastEdge, Souris ErgoClick.

▷ [démonstration live Neo4j](#)

Q3 — Top 3 des best-sellers

```
1 MATCH (:Customer)-[r:PURCHASED]->(p:Product)
2 RETURN p.name AS produit, sum(r.qty) AS unites_vendues
3 ORDER BY unites_vendues DESC LIMIT 3;
```

sum() sur une **propriété de relation** + ORDER BY + LIMIT. Attendu : Aura X **6**, Ecouteurs AirLite **5**, Casque BassPro **4**.

▷ [démonstration live Neo4j](#)

Q4 — Note moyenne par produit

```
1 MATCH (:Customer)-[r:RATED]->(p:Product)
2 RETURN p.name AS produit, round(avg(r.stars),2) AS note, count(r) AS nb_avis
3 ORDER BY note DESC;
```

avg() + count() ensemble. Attendu : ZenBook-14 5.0 (1), BassPro 4.33 (3), Aura X 4.25 (4), Cafetiere 4.0 (1), AirLite 3.5 (2), Pixel-N 2.0 (1).

▷ [démonstration live Neo4j](#)

Q5 — Dépense totale par client

```
1 MATCH (c:Customer)-[r:PURCHASED]->()
2 RETURN c.name AS client, sum(r.qty * r.unitPrice) AS total_depense
3 ORDER BY total_depense DESC;
4
5 // min / max / moyenne GLOBAUX, via WITH :
6 MATCH (c:Customer)-[r:PURCHASED]->()
7 WITH c, sum(r.qty * r.unitPrice) AS t
8 RETURN min(t) AS mini, max(t) AS maxi, round(avg(t),0) AS moyen;
```

Expression dans l'agrégat. Attendu : Bruno 1596 ...Emma 248 (Kenza absente : aucun achat); mini 248, maxi 1596.

▷ démo live Neo4j

Partie 5

**Aller plus loin : OPTIONAL MATCH, hiérarchie,
recommandation**

Q6 — Catégories où un client a acheté

```
1 MATCH (:Customer {name:'Alice'})-[:PURCHASED]->(:Product)
2     -[:IN_CATEGORY]->(cat:Category)
3 RETURN collect(DISTINCT cat.name) AS categories_achetees;
```

collect() agrège des lignes en **liste**; DISTINCT enlève les doublons. Attendu : Alice →
['Smartphones', 'Audio'].

▷ **démo live Neo4j**

MATCH vs OPTIONAL MATCH (pas à pas, 1/2)

```
1 // MATCH classique : exige que le motif existe
2 MATCH (p:Product)
3 MATCH (:Customer)-[r:RATED]->(p)
4 RETURN p.name, count(r) AS nb_avis;
```

Problème : les produits *jamais notés* disparaissent du résultat — le 2^e MATCH exige au moins un avis.
C'est un INNER JOIN.

MATCH vs OPTIONAL MATCH (pas à pas, 2/2)

```
1 MATCH (p:Product)
2 OPTIONAL MATCH (:Customer)-[r:RATED]->(p) // <-- on rend le motif OPTIONNEL
3 RETURN p.name, count(r) AS nb_avis;
```

MATCH = INNER JOIN

les produits sans avis sont absents

OPTIONAL MATCH = LEFT JOIN

tous les produits ; sans avis → 0

OPTIONAL MATCH renvoie NULL quand le motif n'existe pas, et `count(NULL) = 0`. À retenir : « tous les produits, et leurs avis s'ils en ont ». On l'applique tout de suite (Q7/Q8).

▷ **démo live Neo4j**

Q7 — Produits jamais notés (OPTIONAL MATCH)

```
1 MATCH (p:Product)
2 OPTIONAL MATCH (:Customer)-[r:RATED]->(p)
3 WITH p, count(r) AS nb
4 WHERE nb = 0
5 RETURN p.name AS jamais_note ORDER BY p.name;
```

OPTIONAL MATCH = le « LEFT JOIN » : on garde les produits **même sans avis** (r vaut NULL).

Attendu : **7** produits.

▷ [démonstration live Neo4j](#)

Q8 — Clients sans aucun achat

```
1 MATCH (c:Customer)
2 OPTIONAL MATCH (c)-[pu:PURCHASED]->()
3 WITH c, count(pu) AS nb
4 WHERE nb = 0
5 RETURN c.name AS a_relancer;
```

Même schéma, côté client — usage métier : qui **relancer** ? Attendu : **Kenza**.

▷ [démonstration live Neo4j](#)

Q9 — WITH : un pipeline en deux temps

```
1 MATCH (:Customer)-[r:RATED]->(p:Product)
2 WITH p, avg(r.stars) AS note WHERE note >= 4.0
3 MATCH (:Customer)-[pu:PURCHASED]->(p)
4 RETURN p.name AS produit, round(note,2) AS note, sum(pu.qty) AS unites
5 ORDER BY unites DESC;
```

WITH passe un résultat (la note filtrée) à une 2^e interrogation (les ventes) — impossible en un MATCH plat. Attendu : Aura X (4.25, 6), BassPro (4.33, 4), ZenBook-14 (5, 1), Cafetiere (4, 1).

▷ **démo live Neo4j**

Q10 — Catégorie *et* ses sous-catégories (longueur variable)

```
1 MATCH (p:Product)-[:IN_CATEGORY]->(:Category)
2   -[:SUBCATEGORY_OF*0..]->(:Category {name:'Electronique'})
3 RETURN p.name ORDER BY p.name;
```

Le * du Scénario 1, réinvesti sur la **hiérarchie** (*0.. = 0 saut ou plus). Attendu : **10** produits (tout sauf la Cuisine).

▷ **démo live Neo4j**

Q11 — Recommendation (filtrage collaboratif)

```
1 MATCH (moi:Customer {name:'Alice'})-[:PURCHASED]->(commun:Product)
2     <-[:PURCHASED]-(autre:Customer)-[:PURCHASED]->(reco:Product)
3 WHERE autre <> moi AND NOT (moi)-[:PURCHASED]->(reco)
4 RETURN reco.name AS recommandation, count(DISTINCT autre) AS clients_en_commun
5 ORDER BY clients_en_commun DESC, reco.name;
```

« Les clients qui ont acheté pareil → leurs autres achats. » À marteler : autre <> moi, l'exclusion NOT (moi)-[:PURCHASED]->(reco), et count(DISTINCT autre) (compter les **clients**, pas les chemins).

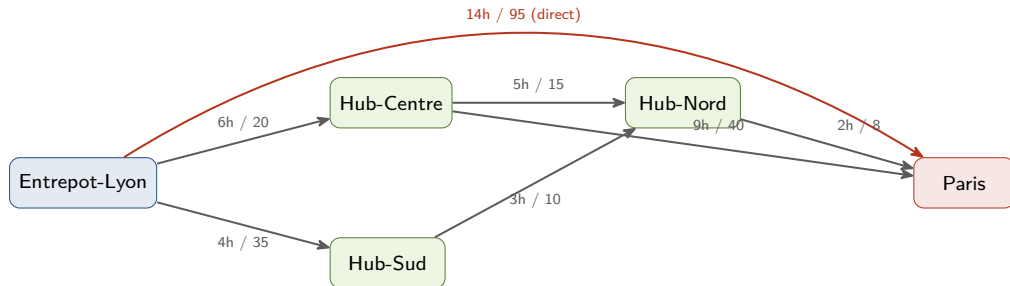
Attendu : **Ecouteurs AirLite 4**, Enceinte SoundOrb 1.

▷ **démo live Neo4j**

Partie 6

Climax : l'itinéraire de livraison

Le sous-graphe logistique (Entrepot-Lyon → Paris)



Livrer *Aura X* (stocké à Lyon) au client *Alice* (à Paris) : plusieurs routes. SHIPS_TO {hours, cost}.

Q12a — shortestPath : le moins de *sauts*

```
1 MATCH (w:Warehouse {name:'Entrepot-Lyon'}), (v:City {name:'Paris'})
2 MATCH p = shortestPath( (w)-[:SHIPS_TO*]->(v) )
3 RETURN [n IN nodes(p) | n.name] AS route, length(p) AS sauts;
```

shortestPath() est **intégré** à Neo4j (pas de plugin). Attendu : [Entrepot-Lyon, Paris], **1 saut**...mais c'est la route directe à **95 € / 14 h !**

▷ **démo live Neo4j**

Comprendre reduce() (pas à pas, 1/2)

```
1 // en pseudo-code :  
2 total = 0  
3 pour chaque x dans [20,15,8] :  
4     total = total + x  
5 // -> total
```

```
1 // en Cypher :  
2 reduce(  
3     total = 0,           // valeur de depart  
4     x IN [20,15,8] |     // on parcourt la  
5         liste            // operation a  
6     total + x           chaque pas  
7 )
```

Déroulé : $total = 0 \rightarrow 0 + 20 = 20 \rightarrow 20 + 15 = 35 \rightarrow 35 + 8 = 43$. `reduce(acc = depart, x IN liste | maj(acc,x))` — sert à *totaliser* / *multiplier* / *concaténer*.

Comprendre reduce() (pas à pas, 2/2) : sur un chemin

```
1 // relationships(p) = la LISTE des aretes du chemin p
2 // on additionne leur propriete .cost :
3 reduce(c = 0, r IN relationships(p) | c + r.cost)
```

Sur la route Lyon→Centre→Nord→Paris, relationships(p) = les 3 arêtes SHIPS_T0 (coûts 20, 15, 8) → reduce = **43 €**. C'est exactement ce qu'on applique maintenant (et ORDER BY choisit le gagnant).

Q12b — coût/durée réels avec reduce

```
1 MATCH p = (w:Warehouse {name:'Entrepot-Lyon'})-[:SHIPS_TO*1..4]->(v:City {name:'Paris'})
2 RETURN [n IN nodes(p) | n.name] AS route,
3         reduce(c=0, r IN relationships(p) | c + r.cost) AS cout_total,
4         reduce(h=0, r IN relationships(p) | h + r.hours) AS duree_totale
5 ORDER BY cout_total ASC;
```

reduce() **somme** une propriété le long du chemin. Le moins cher : Lyon→Centre→Nord→Paris = **43** € (13 h). ORDER BY duree_totale → le plus rapide : Lyon→Sud→Nord→Paris = **9 h** (53 €).

▷ **démo live Neo4j**

Le verdict : 3 critères, 3 itinéraires

Itinéraire	sauts	coût €	heures
shortestPath : Lyon→Paris	1	95	14
Lyon→Centre→Paris	2	60	15
moins cher : Lyon→Centre→Nord→Paris	3	43	13
plus rapide : Lyon→Sud→Nord→Paris	3	53	9

Caveat honnête : `shortestPath()` minimise les *sauts*, pas le coût ni le temps. Pour ceux-là :
`reduce() + ORDER BY`.

Bilan

Ce qu'on a appris

Récapitulatif des clauses

Construire & modifier (CRUD)

- MERGE (+ SET) : trouve-ou-crée, idempotent
- propriétés sur les **relations**
- SET/REMOVE (modifier), DELETE/DETACH DELETE (supprimer)

Agréger / trier

- count, sum, avg, min, max, collect
- ORDER BY, LIMIT, DISTINCT

Aller plus loin

- OPTIONAL MATCH (le « LEFT JOIN »)
- WITH (pipeline)
- *0.. (longueur variable)
- **recommandation** (filtrage collaboratif)

Chemins pondérés

- shortestPath() + reduce()

Le graphe brille pour...et la suite

- **Recommandation** : « qui ressemble à qui » = des motifs courts, pas des dizaines de jointures.
- **Chemins** : itinéraires, dépendances, hiérarchies à profondeur variable.
- **Pour aller plus loin** : la bibliothèque **GDS** (PageRank, communautés, Dijkstra pondéré) — un autre scénario.
- **Supports** : dépôt **GitHub** (PDF + Markdown + scripts Cypher).
- **Examen du 22/06** : théorie des graphes & Neo4j.

Questions ?